SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence COURSE CODE: CSA17

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3)

Assessment Tool Weightage: 50% | Total Marks: 50 | Duration: 1 Hour

Annexure A

Analytical Problem Solving

Name / Reg No: 192524423

Signature of the student: Kamali

This document presents complete, step-by-step solutions to all five questions of Annexure A: (1) the Water Jug problem, (2) PEAS analysis of the Mars Rover, (3) problem formulation and search strategies for the 8-Queens problem, (4) a problem-solving agent with pseudocode for an OLA cab-booking application, and (5) Uniform Cost Search on a warehouse delivery network.

Question 1: The Water Jug Problem

1.1 Problem Understanding

We have two jugs with no measuring markings: a 4-gallon jug (X) and a 3-gallon jug (Y). An unlimited pump is available. The goal is to obtain exactly 2 gallons of water in the 4-gallon jug. The permitted operations are: fill a jug from the pump, empty a jug onto the ground, and pour water from one jug into the other. No other measuring device may be used.

1.2 Problem Formulation

- **State representation:** an ordered pair (x, y), where x is the water in the 4-gallon jug ($0 \le x \le 4$) and y is the water in the 3-gallon jug ($0 \le y \le 3$).
- **Initial state:** (0, 0), both jugs empty.
- **Goal state:** any state (2, y), i.e. $x = 2$.
- **Path cost:** each operation costs 1, so the best solution is the one with the fewest steps.

1.3 Production Rules (Operators)

Rule	Condition	Action	Effect
1	$x < 4$	Fill the 4-gallon jug	$(x, y) \rightarrow (4, y)$
2	$y < 3$	Fill the 3-gallon jug	$(x, y) \rightarrow (x, 3)$
3	$x > 0$	Empty the 4-gallon jug	$(x, y) \rightarrow (0, y)$
4	$y > 0$	Empty the 3-gallon jug	$(x, y) \rightarrow (x, 0)$
5	$x + y \ge 4$ and $y > 0$	Pour from 3-gal into 4-gal until the 4-gal is full	$(x, y) \rightarrow (4, y - (4 - x))$
6	$x + y \ge 3$ and $x > 0$	Pour from 4-gal into 3-gal until the 3-gal is full	$(x, y) \rightarrow (x - (3 - y), 3)$
7	$x + y \le 4$ and $y > 0$	Pour all of the 3-gal into the 4-gal	$(x, y) \rightarrow (x + y, 0)$
8	$x + y \le 3$ and $x > 0$	Pour all of the 4-gal into the 3-gal	$(x, y) \rightarrow (0, x + y)$

1.4 Solution (Breadth-First Search Trace)

Applying the rules from the initial state (0, 0) and exploring level by level (BFS) guarantees the shortest sequence. One shortest solution of 6 steps is shown below:

Step	State (x, y)	Rule applied	Explanation
0	(0, 0)	Initial state	Both jugs are empty.
1	(4, 0)	Rule 1	Fill the 4-gallon jug from the pump.

Step	State (x, y)	Rule applied	Explanation
2	(1, 3)	Rule 6	Pour from the 4-gal into the 3-gal until it is full; 1 gallon is left in the 4-gal.
3	(1, 0)	Rule 4	Empty the 3-gallon jug onto the ground.
4	(0, 1)	Rule 8	Pour the 1 gallon from the 4-gal into the 3-gal.
5	(4, 1)	Rule 1	Fill the 4-gallon jug again.
6	(2, 3)	Rule 6	Pour from the 4-gal into the 3-gal; the 3-gal needs 2 more gallons, leaving exactly 2 in the 4-gal. **Goal reached.**

Alternative 6-step solution: (0,0) → (0,3) → (3,0) → (3,3) → (4,2) → (0,2) → (2,0). Here, the 2 gallons first appear in the 3-gallon jug and are then transferred to the 4-gallon jug.

1.5 Interpretation of Result

Starting from (0, 0), the sequence of six legal operations reaches (2, 3), in which the 4-gallon jug holds exactly 2 gallons, satisfying the goal. The solution works because 3 and 4 are co-prime (gcd = 1), so by repeated filling, emptying and transferring any integer quantity from 1 to 4 can be measured. BFS ensures that no solution with fewer than 6 steps exists, so the answer is optimal under unit step cost. The problem is a classic example of a state-space search where states are small, discrete and fully observable.

Question 2: Mars Rover as an Intelligent Agent

2.1 Introduction

A Mars Rover is an autonomous robotic agent that analyses and explores the Martian surface, collects soil and rock samples, and sends scientific data to Earth. Because radio signals take between 4 and 24 minutes to travel between Earth and Mars, the rover cannot be joystick-controlled in real time and must make many decisions on its own. Its design is best analysed with the PEAS framework (Performance measure, Environment, Actuators, Sensors).

(i) Percepts of the Agent

Percepts are the inputs received through the rover's sensors:

- **Visual percepts:** stereo images from navigation cameras (Navcams, Hazcams) and high-resolution mast cameras showing terrain, rocks, slopes and craters.
- **Position and motion percepts:** wheel odometry, inertial measurement unit (gyroscope and accelerometer) and tilt sensors giving location, heading, speed and slope.
- **Chemical and mineral percepts:** spectrometer data (X-ray, laser-induced breakdown spectroscopy, infrared) describing the composition of rocks and soil.

- **Environmental percepts:** temperature, wind, dust, atmospheric pressure and radiation readings.
- **Internal-state percepts:** battery level, solar or nuclear power output, motor currents, wheel wear, component temperature and memory use.
- **Communication percepts:** commands and updated mission plans uplinked from Earth via orbiters.

(ii) Characterisation of the Operating Environment

Property	Classification	Justification
Observability	Partially observable	Sensors see only a limited range; the terrain behind rocks and under the surface is hidden, and sensor readings are noisy.
Agents	Single agent	The rover acts alone; no competing agents (ground controllers act only as a supervisor).
Determinism	Stochastic	Wheel slip on sand, dust storms and uncertain soil strength mean actions do not always have predictable outcomes.
Episodic vs. sequential	Sequential	Current decisions (path, power use, sample choice) affect later decisions and mission life.
Static vs. dynamic	Dynamic	Dust, lighting, temperature and wind change while the rover is deliberating.
Discrete vs. continuous	Continuous	Positions, angles, time and sensor values vary continuously.
Known vs. unknown	Partly unknown	Physics is known, but the exact local terrain is new and unexplored.

(iii) Actions the Agent Can Take

- Drive forward or backward, steer, turn in place and climb or descend slopes.
- Stop, wait, or enter a safe/sleep mode when power is low or hazards are detected.
- Deploy and move the robotic arm; drill, scoop, abrade and brush rock surfaces.
- Position instruments (spectrometers, microscopic imager) and take measurements.
- Capture, store and seal samples in containers.
- Tilt and rotate the mast camera; take and compress images.
- Transmit data to orbiters or Earth and receive commands.
- Re-orient solar panels or heaters to manage power and temperature.

(iv) Performance Evaluation

The performance measure should be defined in terms of mission outcomes, not merely actions:

Criterion	How it is measured
Scientific value	Number and quality of useful samples analysed; discovery of water, organics or minerals of interest.

Criterion	How it is measured
Area covered	Distance travelled and area explored per unit of time or energy.
Safety and survival	Number of hazard events avoided; avoiding tipping, wheel damage or getting stuck; days of mission life.
Resource efficiency	Energy consumed per measurement; efficient use of data bandwidth and time.
Data quality	Fraction of images and readings successfully transmitted and usable by scientists.
Autonomy	Number of tasks completed without human intervention.

A weighted combination of these criteria, for example Performance = w_1·(science value) + w_2·(coverage) + w_3·(survival) − w_4·(energy used), can be used to compare design alternatives, with survival weighted heavily because a lost rover yields no further science.

(v) Most Suitable Agent Architecture

The most suitable architecture is a **utility-based (goal-based with learning) agent organised in layers.** The reasons are:

- A simple reflex agent is inadequate because the environment is partially observable and the right action depends on history and mission context.
- A model-based reflex agent is needed at the lower level to track the rover's pose, terrain map and internal state (battery, temperature) while some of this is not directly visible.
- A goal-based component is needed because the rover has explicit goals (reach a target rock, collect a sample) and must plan paths using search algorithms such as A*.
- A utility-based component is needed because there are conflicting objectives (science value versus risk versus energy), so the agent must choose actions that maximise expected utility rather than just reach any goal.
- A learning component lets the rover improve terrain classification and traversability estimates from experience, which is valuable in an unknown environment far from human help.

In practice, a layered design is used: a fast reactive layer (obstacle and tilt protection, immediate safety), a deliberative planning layer (path and activity planning) and a supervisory layer (Earth-uploaded goals). This combines the safety of reflexes with the intelligence of planning.

Question 3: 8-Queens Problem – Search Space and Problem Formulation

3.1 Problem Understanding

Eight queens must be placed on an 8×8 chessboard so that no two queens attack one another. A queen attacks any piece in the same row, same column or same diagonal. The configuration given in the question is not a solution because the queen in the first column and the queen in the last column lie on the same diagonal (they

are 7 rows apart and 7 columns apart, so $|\Delta row| = |\Delta col|$). Such a configuration is a **complete-state placement with at least one attacking pair**, which is exactly the kind of state a local-search method must repair.

3.2 Problem Formulation Components

There are two standard formulations.

(a) Incremental formulation (constructive search):

Component	Definition
States	Any arrangement of 0 to 8 queens on the board.
Initial state	Empty board (no queens).
Actions	Add a queen to any empty square.
Transition model	Returns the board with the new queen added to the chosen square.
Goal test	8 queens are on the board and none is attacked.
Path cost	Not important; only the final arrangement matters (every step has cost 1).

This naive version has $64 \times 63 \times \ldots \times 57 \approx 1.8 \times 10^{14}$ sequences to consider, which is far too large.

(b) Improved incremental formulation (one queen per column):

Component	Definition
States	Arrangements of k queens ($0 \le k \le 8$), one in each of the leftmost k columns, with no queen attacking another.
Initial state	No queens on the board.
Actions	Add a queen to the leftmost empty column in a row where it is not attacked by any existing queen.
Transition model	Board with the new queen added in that column.
Goal test	8 queens placed, none attacked.
Path cost	1 per placement (irrelevant to the answer).

This formulation prunes illegal states early and reduces the state space to only **2,057** states, which makes the problem very easy to search.

3.3 Complete-State Formulation (for the given configuration)

Because the given board already has all 8 queens, it is natural to use a complete-state formulation for local search: states are all arrangements with one queen per column ($8^8 \approx 16.7$ million, or $8! = 40{,}320$ if one queen per row and column is enforced); an action moves one queen within its column; the heuristic h is the number of pairs of queens attacking each other. The given board has $h \ge 1$ because of the first-column/last-column diagonal conflict, and the goal is $h = 0$.

3.4 Exploring the Search Space with Search Strategies

Strategy	How it works for 8-Queens	Remarks
Breadth-First Search	Places queens column by column, expanding all legal placements at each depth before going deeper.	Complete, but memory-heavy because every partial board at a level is stored.
Depth-First Search / Backtracking	Places a queen in column 1, then column 2, and so on; when no safe row exists, backtracks to the previous column.	Needs only O(depth) memory; the most practical uninformed method for this problem.
Depth-Limited Search	DFS with a limit of 8, since a solution is always at depth 8.	Avoids infinite paths; the depth is known exactly.
Iterative Deepening	Repeated DFS with increasing limits.	Complete with DFS memory, but redundant here since depth is fixed.
Hill Climbing (local search)	Starting from a complete board, moves a queen to the position that reduces h the most.	Fast, but may be stuck in local minima or plateaus; random restarts help.

Backtracking trace (first few steps): Column 1 → row 1 (safe). Column 2 → rows 1 and 2 are attacked, so row 3 is chosen. Column 3 → row 5. Column 4 → row 2 … and so on. If a column has no safe row, the algorithm returns to the previous column and tries its next row. Continuing, the first solution found is rows (1, 5, 8, 6, 3, 7, 2, 4) for columns 1 to 8.

3.5 A Valid Solution and Its Verification

The arrangement with queens at (column, row) = (1,1), (2,5), (3,8), (4,6), (5,3), (6,7), (7,2), (8,4) is valid: all rows are distinct; the values (row − column) are 0, 3, 5, 2, −2, 1, −5, −4 (all distinct, so no shared diagonal of one direction); and the values (row + column) are 2, 7, 11, 10, 8, 13, 9, 12 (all distinct, so no shared diagonal of the other direction).

3.6 Interpretation

The 8-Queens problem has 92 distinct solutions, which reduce to 12 fundamental solutions after removing rotations and reflections. A good problem formulation drastically reduces the search space: from about 1.8×10^{14} for the naive version to 2,057 states for the one-queen-per-column version. The key lesson is that choosing the right state representation, together with early constraint checking, is often more important than the search algorithm used.

Question 4: OLA Cab Booking – Problem-Solving Agent

4.1 Problem Understanding

A customer wishes to travel from a source location to a destination using the OLA app and may choose among cab types such as Micro, Mini, Prime Sedan, Prime SUV, Shared and Auto. The choice depends on comfort,

cost, waiting time and number of passengers. The system must (1) understand the request, (2) find nearby available cabs, (3) calculate route, fare and arrival time, (4) present choices and (5) complete the booking and the trip.

4.2 Type of Agent

The most appropriate design is a **goal-based problem-solving agent with a utility-based choice step.** Reasons:

- It has a clear goal: reach the destination from the source safely.
- It formulates the problem (initial state = pickup location, goal state = destination, actions = choose a cab type and a route, path cost = fare + time) and then searches for a sequence of actions that achieves the goal.
- The cab type choice involves trade-offs (price, comfort, waiting time), which is handled by a utility function that ranks the cab types, personalised to the customer's preferences.
- The environment is dynamic (traffic and cab availability change), so the agent re-plans when a driver cancels or a road is blocked.

4.3 PEAS Description

Element	Description
Performance	Low waiting time, low fare, shortest travel time, customer comfort and safety, high satisfaction rating.
Environment	City road network, traffic, weather, available drivers and cab types, customer, payment gateway.
Actuators	App screens (list of cabs and fare), booking confirmation, driver assignment, notifications, payment processing, navigation.
Sensors	GPS location, customer input (source, destination, passengers, preferences), driver availability and live traffic data.

4.4 Problem Formulation

- **Initial state:** customer at pickup location, no cab booked.
- **Actions:** SELECT-CAB-TYPE(t), CONFIRM-BOOKING, TRACK-DRIVER, START-TRIP, FOLLOW-ROUTE, END-TRIP, PAY.
- **Transition model:** each action changes the booking state (searching → booked → driver arrived → trip started → destination reached → paid).
- **Goal test:** customer is at the destination and the payment is completed.
- **Path cost:** weighted combination of fare, waiting time and travel time.

4.5 Pseudocode

```
function OLA-BOOKING-AGENT(percept) returns an action
    persistent: seq (action sequence, initially empty), state, goal, problem

    state ← UPDATE-STATE(state, percept)        // GPS, source, destination, traffic
    if seq is empty then
```

```
        goal ← FORMULATE-GOAL(state)            // reach destination
        problem ← FORMULATE-PROBLEM(state, goal)
        cabList ← FIND-AVAILABLE-CABS(source)    // mini, micro, sedan, shared, prime…
        if cabList is empty then return NO-CAB-AVAILABLE message
        for each cab type t in cabList do
            route[t] ← SEARCH(problem, source, destination)   // e.g. A* on road graph
            fare[t] ← COMPUTE-FARE(t, route[t])
            eta[t] ← ESTIMATE-ARRIVAL(t, source)
            U[t] ← UTILITY(fare[t], eta[t], comfort[t], user-preferences)
        end for
        DISPLAY(cabList, fare, eta)
        chosen ← USER-SELECTION(cabList)          // or best U[t] as default
        seq ← [CONFIRM-BOOKING(chosen), ASSIGN-DRIVER, TRACK-DRIVER,
            START-TRIP, FOLLOW-ROUTE(route[chosen]), END-TRIP, PAY]
    end if
    action ← FIRST(seq)
    seq ← REST(seq)
    if action fails (driver cancels or road blocked) then
        seq ← empty                             // re-plan with new percept
    return action
```

4.6 Interpretation

The agent follows the classic cycle of formulate goal → formulate problem → search → execute. Because OLA operates in a dynamic, partially observable and stochastic environment (traffic and driver behaviour are not fully predictable), the plan is executed one action at a time with monitoring, and the agent re-plans whenever the percept shows the plan is no longer valid. Using utility values to rank cab types enables personalised recommendations, such as showing Shared for cost-conscious users and Prime for comfort-seeking users.

Question 5: Uniform Cost Search on the Delivery Network

5.1 Problem Understanding

We must find the least-cost path from warehouse S to warehouse G in a weighted directed graph using Uniform Cost Search (UCS). UCS always expands the node in the frontier (priority queue) having the lowest path cost g(n) from the start node. It is complete when step costs are positive and is **optimal** because the first time the goal is removed from the queue, its cost is the minimum possible.

5.2 Graph Edges (from the figure)

From	To	Cost
S	A	1
S	G	12
A	B	3
A	C	1
B	D	3
C	D	1
C	G	2
D	G	3

5.3 UCS Algorithm

1. Initialise frontier as a priority queue with (S, cost 0); explored ← { }
2. Loop:
    a. If the frontier is empty → failure.
    b. Remove the node n with the lowest g(n).
    c. If n is the goal G → return the path and cost.
    d. Add n to explored.
    e. For each neighbour m of n with edge cost c(n, m):
        if m is not in explored or frontier → insert m with g = g(n) + c
        else if m is in frontier with higher g → replace with the lower g

5.4 Step-by-Step Execution

Step	Node expanded (g)	Successors generated (path cost)	Frontier after step (sorted by cost)
1	S (0)	A = 0+1 = 1; G = 0+12 = 12	A(1), G(12)
2	A (1)	B = 1+3 = 4; C = 1+1 = 2	C(2), B(4), G(12)
3	C (2)	D = 2+1 = 3; G = 2+2 = 4 (better than 12, so G is updated)	D(3), B(4), G(4)
4	D (3)	G = 3+3 = 6 (worse than 4, ignored)	B(4), G(4)
5	B (4)	D = 4+3 = 7 (D already explored with 3, ignored)	G(4)
6	G (4)	Goal removed from the frontier, so search stops.	Goal reached

(In step 5, B and G both have cost 4; with first-in-first-out tie-breaking B is expanded first, and it does not change the result. Either tie-break order yields the same final answer.)

5.5 Result

Optimal path: S → A → C → G with total cost = 1 + 1 + 2 = 4.

Candidate path	Total cost	Remarks
S → A → C → G	1 + 1 + 2 = 4	**Optimal (least cost)**
S → A → C → D → G	1 + 1 + 1 + 3 = 6	Costlier
S → A → B → D → G	1 + 3 + 3 + 3 = 10	Costlier
S → G (direct)	12	Shortest in hops but costliest

5.6 Interpretation and Analysis

The direct route S → G has the fewest edges but the highest cost (12); the three-leg route through A and C costs only 4, which is a saving of 8 units (about 67%) in fuel and time. This shows why breadth-first search (which minimises number of edges) would not be appropriate for a cost-sensitive logistics network, while UCS finds the true minimum-cost route. UCS expanded the nodes S, A, C, D, B in order of increasing cost and never needed to expand nodes beyond cost 4.

Properties of UCS: complete (if all edge costs $\geq \varepsilon > 0$); optimal; time and space complexity $O(b^{(1 + \lfloor C^*/\varepsilon \rfloor)})$, where C^* is the optimal cost, b is the branching factor and ε is the minimum edge cost. For large real-world networks, UCS can be improved by using A* with a heuristic such as straight-line distance between warehouses.

Conclusion: the logistics company should use the route S → A → C → G with a total cost of 4 units to deliver packages from S to G.